

Operator của `SmallCNN` — bộ tài liệu đầy đủ

Bước 2 trong phương pháp Thực chiến của Thầy: **viết operator + mã giả** cho mọi phép toán trên đường inference, làm đầu vào cho golden model C (Phase 2D).

Đường inference đầy đủ

Sau khi train xong, model chạy đúng 12 bước này và không có gì khác:

```
ảnh 1×28×28
├─ conv2d (1→8, k3, p1) → 8×28×28
├─ relu → 8×28×28
├─ maxpool (k2, s2) → 8×14×14
├─ conv2d (8→16, k3, p1) → 16×14×14
├─ relu → 16×14×14
├─ maxpool (k2, s2) → 16× 7× 7
├─ conv2d (16→16, k3, p1) → 16× 7× 7
├─ relu → 16× 7× 7
├─ maxpool (k2, s2) → 16× 3× 3
├─ flatten → 144
├─ linear (144→10) → 10 logit
└─ argmax → 1 chữ số
```

Chỉ có 6 operator khác nhau. Không có softmax, không có loss function, không có optimizer — những thứ đó chỉ tồn tại lúc train.

Bảng tổng hợp

Operator	File	Params	Phép tính / ảnh	FPGA
Conv2d	conv2d.md	3.568	395.136 MAC	✅ Rất tốt
ReLU	relu.md	0	10.192 so sánh	✅ Tốt nhất
MaxPool2d	maxpool2d.md	0	7.488 so sánh	✅ Rất tốt
Flatten	flatten.md	0	0	✅ Miễn phí
Linear	linear.md	1.450	1.440 MAC	⚠️ Nghẽn băng thông
argmax	argmax.md	0	9 so sánh	✅ Gần như miễn phí
	Tổng	5.018	~414.000	

Ba lớp conv chiếm **95%** khối lượng tính toán nhưng chỉ **71%** tham số. Lớp `fc` ngược lại: 0,3% tính toán nhưng 29% tham số. Hai trục khác nhau, tối ưu khác nhau.

Định dạng chung

Mỗi file có đúng 7 mục:

1. **Vai trò** — operator này tồn tại để làm gì
2. **Công thức** — kèm cách suy ra, không chỉ chép kết quả
3. **Mã giả** — viết được thành C ngay, không phụ thuộc thư viện
4. ****Áp dụng vào `SmallCNN`**** — số cụ thể của model này
5. **Mức FPGA-friendly** — phép toán nào tốn phần cứng
6. **Chú ý khi port sang C** — bẫy làm sai mà không báo lỗi
7. **Tự kiểm tra** — câu hỏi làm trên giấy, **kèm đáp án đầy đủ** (cập nhật 23/09)

Ba bẫy nguy hiểm nhất khi viết golden model C

Cả ba đều **không crash**, chỉ cho kết quả sai:

1. ****Layout NCHW vs NHWC ở `flatten`**** — vector 144 bị hoán vị, accuracy rơi về ~10%. Cách bắt: in vector 144 từ PyTorch và từ C cho cùng một ảnh, so từng phần tử. → [flatten.md](#) mục 6
2. **Thứ tự chiều trọng số** — PyTorch `[out][in]`, Keras `[in][out]`. Ngược nhau. → [conv2d.md](#) và [linear.md](#) mục 6
3. ****Khởi tạo accumulator bằng 0 thay vì `bias`**** — lệch đúng bằng bias, rất khó nhìn ra. → có ở cả `conv2d` và `linear`

Bẫy thứ tư, chỉ lộ ra khi có hai logit bằng nhau:

1. ****Viết `>=` thay vì `>` trong `argmax`**** — trả về chỉ số lớn nhất thay vì nhỏ nhất, lệch với `torch.argmax`. Ở float32 chưa ảnh hưởng ảnh nào (đo được: 0/10.000), nhưng khi logit thành số nguyên thì có **7/10.000 ảnh** đổi kết quả. Accuracy có thể **không đổi**, nên lỗi này vô hình nếu chỉ so accuracy. → [argmax.md](#) mục 6 và 7

Hai chỗ bỏ bớt được phép toán cho FPGA

Đây là phần trả lời trực tiếp yêu cầu "thay/bỏ phép toán khó" trong phạm vi project:

Bỏ gì	Tiết kiệm	Vì sao đúng
Softmax ở lớp cuối	10 phép <code>exp()</code> + 1 phép chia	<code>exp</code> đơn điệu tăng nên không đổi thứ tự → <code>argmax</code> ra kết quả y hệt. argmax.md mục 2
Đổi thứ tự **<code>relu</code> ↔ <code>maxpool</code>**	10.192 → 2.496 phép ReLU (4,08 lần)	<code>max</code> và <code>relu</code> đều đơn điệu không giảm nên hoán vị được. Chỉ đúng với MaxPool, sai với AvgPool. relu.md mục 5

Cả hai đều cho kết quả **giống hệt bit-for-bit**, không phải xấp xỉ — khác với việc thay `Linear` bằng `Conv2d` (xem [linear.md](#) mục 7 câu 4), chỗ đó chỉ tương đương về mặt toán học và vẫn lệch `2,174e-`

04 do thứ tự cộng dồn float32.

Biên an toàn của lớp cuối

Đo bằng `study/dem_hoa_logit.py`: khoảng cách nhỏ nhất giữa hai logit cao nhất trên 10.000 ảnh test là `**1,900e-02**`. Mọi sai số của golden model C nhỏ hơn hẳn con số đó thì không thể lật một dự đoán nào. Dưới `1e-03` là an toàn tuyệt đối. Chi tiết ở `roadmap-digit-recognition-v3.md` mục 6.

Hai chỗ đã sửa lại (23/09)

- `**relu.md` mục 1 từng ghi ReLU là "nguồn phi tuyến duy nhất" của `SmallCNN`. Sai — `MaxPool2d` cũng phi tuyến (`max(a,b)` không tuyến tính). Bỏ hết ReLU thì mạng **không** sụp về một lớp tuyến tính, chỉ yếu đi nhiều. Đã sửa, kèm đáp án Câu 1 mục 7.

- `**flatten.md` mục 2** từng ghi `(c=5, y=1, x=2)` ở chỉ số **47**, trong khi chính cột "Tính" bên cạnh ghi `45 + 3 + 2 = 50`. Đã sửa thành 50.

Chưa làm

- Lượng tử hoá từng operator (Phase 2E, làm **sau** golden model C float32).

Kết quả khảo sát ban đầu ở `study/xem_quantization.py`.

- Operator cho phần train (`CrossEntropyLoss`, `Adam`) — **không nằm trên đường inference**, không cần cho chip.